

Sistema de Gestão de Territórios e Publicações (GTP)

Documento 11 – Modelo Entidade-Relacionamento (DER)

Projeto: Sistema de Gestão de Territórios e Publicações (GTP)

Versão: 1.0.0

Status: Em Elaboração

Data: Julho/2026

Autor: Fabio André Zatta

1 – Introdução

1.1 Objetivo

Este documento apresenta o **Modelo Entidade-Relacionamento (DER)** do sistema **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é definir a estrutura lógica dos dados que compõem o sistema, descrevendo as entidades de negócio, seus atributos, relacionamentos, cardinalidades, restrições de integridade e diretrizes para implementação no banco de dados PostgreSQL.

O modelo descrito neste documento constitui a base para a construção da camada de persistência da aplicação, orientando a implementação das entidades JPA/Hibernate, dos scripts de migração do Flyway, dos repositórios Spring Data JPA e dos serviços responsáveis pela manipulação das informações.

Além disso, o DER garante consistência entre os requisitos funcionais definidos nos documentos anteriores e a estrutura física do banco de dados.

1.2 Objetivos Específicos

Este documento busca:

- identificar todas as entidades do domínio;
- definir atributos e chaves primárias;
- estabelecer relacionamentos entre entidades;
- especificar cardinalidades;
- definir restrições de integridade;
- orientar a implementação das entidades JPA;
- servir de referência para o banco PostgreSQL;
- apoiar a geração dos diagramas de banco de dados;
- facilitar manutenção e evolução do modelo.

1.3 Escopo

O Modelo Entidade-Relacionamento contempla todos os módulos do GTP, incluindo:

- autenticação;
- usuários;
- perfis e permissões;
- congregações;
- territórios;

- quadras;
- endereços;
- publicadores;
- designações;
- visitas;
- estudos bíblicos;
- publicações;
- notificações;
- auditoria;
- configurações;
- relatórios;
- indicadores.

Cada módulo será representado por um conjunto de entidades relacionadas entre si.

1.4 Objetivos do Modelo

O modelo deverá atender aos seguintes requisitos:

- eliminar redundâncias desnecessárias;
- garantir integridade referencial;
- favorecer consultas eficientes;
- suportar crescimento da base de dados;
- permitir evolução do domínio sem impactos significativos;
- facilitar implementação utilizando JPA/Hibernate.

1.5 Princípios de Modelagem

A modelagem do banco seguirá os seguintes princípios:

Normalização

As entidades deverão estar normalizadas até, no mínimo, a **Terceira Forma Normal (3FN)**.

Quando necessário, poderão ser aplicadas técnicas controladas de desnormalização visando desempenho, desde que documentadas.

Integridade Referencial

Todos os relacionamentos utilizarão chaves estrangeiras devidamente definidas.

Não serão permitidos registros órfãos em entidades críticas.

Integridade de Domínio

Os dados armazenados deverão respeitar:

- regras de negócio;
- tipos de dados;
- obrigatoriedade dos campos;
- restrições de unicidade;
- validações de domínio.

Independência da Aplicação

O modelo de dados deverá representar o domínio do negócio e não detalhes específicos da implementação da aplicação.

Evolução Controlada

Novas entidades poderão ser adicionadas futuramente preservando a compatibilidade com o modelo existente.

Todas as alterações estruturais deverão ser realizadas por meio de migrações versionadas utilizando Flyway.

1.6 Convenções de Nomenclatura

Para garantir padronização, serão adotadas as seguintes convenções.

Tabelas

- nomes no singular;
- letras minúsculas;
- separação por underscore (`snake_case`).

Exemplos:

```
usuario
publicador
territorio
estudo_biblico
auditoria
```

Colunas

Também utilizarão o padrão snake_case.

Exemplos:

```
nome_completo
data_cadastro
usuario_id
congregacao_id
ultima_atualizacao
```

Chaves Primárias

Todas as tabelas possuirão uma chave primária denominada:

```
id
```

Tipo recomendado:

```
BIGINT
```

Gerado automaticamente pelo PostgreSQL utilizando identidade (GENERATED BY DEFAULT AS IDENTITY).

Chaves Estrangeiras

As referências seguirão o padrão:

```
entidade_id
```

Exemplos:

```
usuario_id
territorio_id
congregacao_id
publicador_id
```

1.7 Tecnologias Relacionadas

Este modelo será implementado utilizando:

Tecnologia	Finalidade
PostgreSQL 17	Banco de dados relacional.
Spring Data JPA	Persistência dos dados.
Hibernate ORM	Mapeamento objeto-relacional (ORM).
Flyway	Controle de versão do banco de dados.
Jakarta Persistence	Especificação JPA.
Jakarta Validation	Validação dos dados.

1.8 Relação com os Demais Documentos

Documento	Relação
Documento 03 – Regras de Negócio	Define as regras que influenciam o modelo de dados.
Documento 04 – Casos de Uso	Identifica as entidades manipuladas pelo sistema.
Documento 05 – Arquitetura do Sistema	Estabelece a arquitetura na qual o modelo será utilizado.
Documento 06 – Arquitetura Backend	Implementará as entidades e repositórios JPA.
Documento 10 – API REST	Utiliza as entidades como base para os contratos da API.

1.9 Conclusão do Capítulo

O Modelo Entidade-Relacionamento representa o alicerce da camada de persistência do GTP. A definição cuidadosa das entidades, seus atributos e relacionamentos garante integridade, desempenho e escalabilidade, além de assegurar que as regras de negócio estabelecidas nos documentos anteriores sejam refletidas de forma consistente no banco de dados.

A adoção de convenções padronizadas, normalização adequada e integração com tecnologias como PostgreSQL, Hibernate e Flyway permitirá uma implementação robusta, preparada para a evolução contínua da plataforma.

2 – Diretrizes Gerais da Modelagem

2.1 Objetivo

Este capítulo define os princípios e padrões utilizados na construção do Modelo Entidade-Relacionamento (DER) do GTP.

As diretrizes aqui apresentadas têm como objetivos:

- padronizar a modelagem de dados;
- garantir integridade referencial;
- facilitar a implementação com JPA/Hibernate;
- promover desempenho e escalabilidade;
- simplificar a manutenção e evolução do banco de dados.

Todas as entidades do sistema deverão respeitar essas diretrizes.

2.2 Arquitetura Lógica do Banco de Dados

O banco de dados do GTP será estruturado em torno dos principais domínios do negócio, organizados de forma modular.

Domínios principais

- Segurança e Autenticação
- Administração
- Congregações
- Territórios
- Publicadores
- Designações
- Visitas
- Estudos Bíblicos
- Publicações
- Configurações
- Relatórios
- Auditoria

Cada domínio agrupa entidades relacionadas funcionalmente, favorecendo a organização do modelo e a separação de responsabilidades.

2.3 Normalização

O modelo deverá seguir, como regra geral, a **Terceira Forma Normal (3FN)**.

Os objetivos da normalização são:

- eliminar redundâncias;
- evitar inconsistências;
- reduzir anomalias de atualização;
- facilitar a manutenção.

Em situações específicas, poderá ser adotada desnormalização controlada para otimização de desempenho, desde que tecnicamente justificada e documentada.

2.4 Estratégia de Identificação das Entidades

Todas as entidades possuirão uma chave primária denominada:

`id`

Características:

- tipo `BIGINT`;
- geração automática pelo PostgreSQL;
- imutável;
- utilizada como identificador técnico da entidade.

Exemplo SQL:

```
id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY
```

2.5 Chaves Naturais e Unicidade

Além da chave primária, algumas entidades possuirão identificadores naturais que deverão ser únicos.

Exemplos:

Entidade	Campo
Usuário	e-mail
Congregação	código
Território	número
Publicador	matrícula (quando aplicável)

Esses campos deverão possuir restrições `UNIQUE`.

2.6 Relacionamentos

O modelo utilizará três tipos principais de relacionamentos.

Um para Um (1:1)

Aplicado quando duas entidades possuem dependência exclusiva.

Exemplo:

- Usuário ↔ Preferências de Usuário.

Um para Muitos (1:N)

Relacionamento predominante no sistema.

Exemplos:

- Congregação → Publicadores
- Congregação → Territórios
- Território → Endereços
- Publicador → Designações

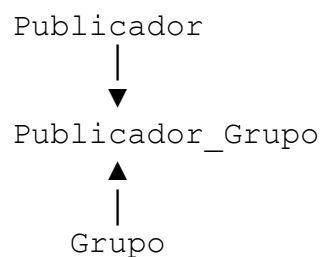
Muitos para Muitos (N:N)

Quando necessário, será implementado por meio de entidades associativas.

Exemplo:

Publicador ↔ Grupo de Serviço

Representação:



Essa abordagem facilita a inclusão de atributos adicionais na relação, como data de ingresso, função ou status.

2.7 Cardinalidades

As cardinalidades deverão ser explicitamente definidas para todos os relacionamentos.

Exemplo:

Relacionamento	Cardinalidade
Congregação → Publicador	1:N
Congregação → Território	1:N
Território → Endereço	1:N
Publicador → Designação	1:N
Usuário → Perfil	N:1

Essas definições orientarão tanto o DER quanto o mapeamento JPA.

2.8 Integridade Referencial

Todos os relacionamentos deverão ser protegidos por chaves estrangeiras (`FOREIGN KEY`).

Regras gerais:

- não permitir referências inexistentes;
- impedir registros órfãos;
- garantir consistência entre tabelas.

As políticas de exclusão (`ON DELETE`) e atualização (`ON UPDATE`) deverão ser definidas conforme a regra de negócio de cada relacionamento.

2.9 Campos Comuns

Grande parte das entidades compartilhará um conjunto padronizado de campos.

Campo	Finalidade
id	Identificador único
ativo	Indica se o registro está ativo
data_criacao	Data de criação
data_atualizacao	Última alteração
criado_por	Usuário responsável pela criação
atualizado_por	Usuário responsável pela alteração
versao	Controle de concorrência otimista

Esses campos facilitam auditoria, rastreabilidade e controle de alterações.

2.10 Auditoria

As entidades críticas deverão registrar informações de auditoria.

Campos mínimos:

- data de criação;
- usuário criador;
- data da última alteração;
- usuário responsável pela alteração.

Além disso, operações relevantes serão registradas em tabela específica de auditoria, conforme definido no Documento 08 – Segurança.

2.11 Soft Delete

Sempre que possível, será adotada a estratégia de exclusão lógica (*soft delete*).

Em vez de remover fisicamente um registro, será utilizado um indicador de inativação.

Exemplo:

Campo Valor

ativo false

Vantagens:

- preservação do histórico;
- recuperação de registros;
- manutenção da integridade referencial;
- suporte à auditoria.

A exclusão física será reservada para situações específicas, como dados temporários ou exigências legais.

2.12 Controle de Concorrência

Será adotado o mecanismo de **Concorrência Otimista** utilizando um campo de versão.

Exemplo:

```
versao
```

Esse campo será incrementado automaticamente a cada atualização, evitando sobrescritas simultâneas.

No JPA, será implementado por meio da anotação `@Version`.

2.13 Convenções para Tipos de Dados

Os tipos de dados deverão seguir uma padronização.

Informação	Tipo PostgreSQL
Identificador	BIGINT
Texto curto	VARCHAR
Texto longo	TEXT
Data	DATE
Data e hora	TIMESTAMP WITH TIME ZONE
Booleano	BOOLEAN
Valor monetário	NUMERIC(12,2)
Enumeração	VARCHAR ou ENUM (conforme decisão arquitetural)

Essa padronização facilita o mapeamento para Java e evita inconsistências.

2.14 Índices

Os índices deverão ser criados considerando:

- campos utilizados em pesquisas frequentes;
- chaves estrangeiras;
- colunas de ordenação;
- campos únicos.

Exemplos:

- e-mail;
- número do território;
- código da congregação;
- CPF (quando utilizado);
- datas de designação.

A criação excessiva de índices deverá ser evitada para não comprometer operações de escrita.

2.15 Convenções de Nomenclatura

Serão adotadas as seguintes convenções.

Tabelas

- singular;
- minúsculas;
- snake_case.

Exemplo:

```
publicador
territorio
congregacao
```

Colunas

Também em snake_case.

Exemplo:

```
nome_completo
data_designacao
usuario_id
```

Índices

Prefixo idx_.

Exemplo:

```
idx_publicador_nome
idx_territorio_numero
```

Chaves estrangeiras

Prefixo fk_.

Exemplo:

```
fk_publicador_congregacao
```

2.16 Estratégia para Migrações

Toda alteração estrutural no banco de dados deverá ser realizada exclusivamente por meio de migrações versionadas utilizando Flyway.

Princípios:

- scripts imutáveis;
- versionamento sequencial;
- rastreabilidade das alterações;
- execução automática nos ambientes.

2.17 Boas Práticas

Durante a modelagem deverão ser observadas as seguintes recomendações:

- evitar duplicidade de informações;
- utilizar nomes claros e padronizados;
- documentar todas as entidades;
- definir corretamente cardinalidades;
- utilizar restrições de integridade;
- prever evolução futura do modelo;
- manter consistência entre DER, entidades JPA e migrações Flyway.

2.18 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura geral da solução.
Documento 06 – Arquitetura Backend	Implementa as entidades e repositórios conforme este modelo.
Documento 08 – Segurança	Define requisitos de auditoria e controle de acesso refletidos no modelo de dados.
Documento 09 – Docker	Provê a infraestrutura do PostgreSQL utilizada pelo modelo.
Documento 10 – API REST	Consome as entidades por meio dos contratos da API.

2.19 Conclusão do Capítulo

As diretrizes apresentadas neste capítulo estabelecem os fundamentos para a construção de um modelo de dados consistente, seguro e preparado para evolução. A adoção de

princípios como normalização, integridade referencial, auditoria, *soft delete*, concorrência otimista e versionamento do esquema garante que o banco de dados do GTP suporte de forma eficiente as regras de negócio e os requisitos de desempenho da plataforma.

Essas definições servirão como referência para todos os capítulos seguintes, nos quais serão detalhadas as entidades, seus atributos e relacionamentos.

3 – Entidades do Sistema

3.1 Objetivo

Este capítulo apresenta a especificação conceitual das entidades que compõem o domínio do Gestor de Territórios e Publicações (GTP).

Para cada entidade são definidos:

- finalidade;
- atributos;
- chave primária;
- chaves estrangeiras;
- restrições de integridade;
- relacionamentos;
- observações de implementação.

As definições aqui descritas servirão como referência para a implementação das entidades JPA/Hibernate e para a criação das tabelas no PostgreSQL.

3.2 Organização das Entidades

As entidades estão agrupadas conforme os módulos do sistema.

Domínio	Principais Entidades
Segurança	Usuário, Perfil, Permissão, RefreshToken
Administração	Configuração, Parâmetro, Auditoria
Congregações	Congregação
Territórios	Território, Quadra, Endereço
Publicadores	Publicador, Grupo de Serviço
Designações	Designação, Histórico de Designação
Visitas	Visita
Estudos Bíblicos	Estudo Bíblico
Publicações	Publicação
Notificações	Notificação

Domínio	Principais Entidades
Dashboard	Visão Analítica (Views)

3.3 Entidade: Usuário

Finalidade

Representa os usuários autenticados no sistema.

Tabela

usuario

Principais atributos

Campo	Tipo	Obrigatório
id	BIGINT	Sim
nome	VARCHAR(150)	Sim
email	VARCHAR(150)	Sim
senha	VARCHAR(255)	Sim
ativo	BOOLEAN	Sim
ultimo_login	TIMESTAMP	Não
data_criacao	TIMESTAMP	Sim
data_atualizacao	TIMESTAMP	Sim
versao	BIGINT	Sim

Restrições

- e-mail único;
- senha criptografada;
- usuário não pode existir sem perfil.

Relacionamentos

- Perfil (N:1)
- Congregação (N:1)
- Auditoria (1:N)
- Refresh Token (1:N)

3.4 Entidade: Perfil

Finalidade

Define os papéis de acesso ao sistema.

Tabela

perfil

Atributos

Campo	Tipo
id	BIGINT
nome	VARCHAR(80)
descricao	TEXT

Relacionamentos

- Usuários (1:N)
- Permissões (N:N)

3.5 Entidade: Permissão

Finalidade

Representa permissões específicas do sistema.

Tabela

permissao

Campos principais

- id
- nome
- codigo
- descricao

Relacionamento

Perfil ↔ Permissão

N:N

Implementado através da entidade associativa:

`perfil_permissao`

3.6 Entidade: Refresh Token

Finalidade

Controle de renovação de autenticação.

Tabela

`refresh_token`

Campos

- id
- token
- expiracao
- revogado
- usuario_id

Relacionamento

Usuário 1:N RefreshToken

3.7 Entidade: Congregação

Tabela

`congregacao`

Campos principais

Campo	Tipo
id	BIGINT
nome	VARCHAR
codigo	VARCHAR
cidade	VARCHAR
estado	VARCHAR
ativa	BOOLEAN

Relacionamentos

- Publicadores
- Territórios
- Usuários
- Configurações

3.8 Entidade: Território

Tabela

territorio

Campos

- id
- numero
- descricao
- status
- data_criacao
- congregacao_id
- quadra_id

Relacionamentos

- Congregação
- Quadra
- Endereços
- Designações
- Visitas

3.9 Entidade: Quadra

Tabela

quadra

Campos

- id
- nome
- descrição

Relacionamentos

- Territórios

3.10 Entidade: Endereço

Tabela

endereco

Campos

- id
- logradouro
- numero
- complemento
- bairro
- cidade
- CEP
- território_id

Relacionamentos

- Território

3.11 Entidade: Publicador

Representa o publicador pertencente à congregação.

Tabela

publicador

Campos principais

Campo	Tipo
id	BIGINT
nome_completo	VARCHAR
data_nascimento	DATE
telefone	VARCHAR
email	VARCHAR
sexo	ENUM
ativo	BOOLEAN

Campo	Tipo
congregacao_id	BIGINT

Relacionamentos

- Congregação
- Grupo de Serviço
- Designações
- Visitas
- Estudos Bíblicos

3.12 Entidade: Grupo de Serviço

Tabela

grupo_servico

Campos

- id
- nome
- dirigente
- congregacao_id

Relacionamentos

Grupo
↓
Publicadores

3.13 Entidade: Designação

Representa a entrega de um território.

Tabela

designacao

Campos

- id
- data_designacao
- data_prevista
- data_devolucao
- status

- observacao

Relacionamentos

- Território
- Publicador

3.14 Entidade: Histórico de Designação

Tabela

historico_designacao

Objetivo

Manter todas as alterações realizadas durante o ciclo de vida de uma designação.

Relacionamentos

Designação
↓
Histórico

3.15 Entidade: Visita

Tabela

visita

Campos

- data
- resultado
- observação
- território
- publicador

3.16 Entidade: Estudo Bíblico

Tabela

`estudo_biblico`

Campos

- estudante
- endereço
- dia_semana
- horário
- status

Relacionamentos

Publicador

↓

Estudo

3.17 Entidade: Publicação

Tabela

`publicacao`

Campos

- título
- tipo
- idioma
- quantidade

3.18 Entidade: Configuração

Tabela

`configuracao`

Campos

- chave
- valor
- descrição

3.19 Entidade: Auditoria

Tabela

`auditoria`

Campos principais

- usuário
- data
- ação
- recurso
- IP
- navegador
- detalhes

Essa entidade registra todas as operações relevantes executadas no sistema.

3.20 Entidade: Notificação

Tabela

`notificacao`

Campos

- título
- mensagem
- tipo
- data_envio
- lida

Relacionamentos

Usuário
↓
Notificações

3.21 Entidades Auxiliares

Além das entidades principais, o sistema poderá possuir entidades auxiliares para apoio ao domínio.

Exemplos:

- perfil_permissao
- publicador_grupo
- configuracao_global
- parametro_sistema
- log_integracao
- arquivo_importacao
- historico_login

Essas entidades não representam objetos centrais do negócio, mas oferecem suporte à infraestrutura e às funcionalidades administrativas.

3.22 Campos Padronizados

Grande parte das entidades compartilhará os seguintes atributos:

Campo	Finalidade
id	Identificador
ativo	Exclusão lógica
data_criacao	Auditoria
data_atualizacao	Auditoria
criado_por	Auditoria
atualizado_por	Auditoria
versao	Controle de concorrência

Esses campos serão implementados por meio de uma superclasse base (`BaseEntity`) no backend, promovendo reutilização e padronização.

3.23 Resumo das Entidades

Domínio	Quantidade Estimada
Segurança	4
Administração	4
Congregações	1
Territórios	3
Publicadores	2
Designações	2
Visitas	1
Estudos Bíblicos	1
Publicações	1

Domínio	Quantidade Estimada
Notificações	1
Entidades Auxiliares	8 a 12

Total estimado: aproximadamente **30 a 35 entidades**, número adequado para um sistema com a complexidade prevista para o GTP.

3.24 Relação com os Demais Documentos

Documento	Relação
Documento 03 – Regras de Negócio	Define as regras aplicáveis às entidades.
Documento 04 – Casos de Uso	Identifica as operações realizadas sobre as entidades.
Documento 06 – Arquitetura Backend	Implementa as entidades JPA e os repositórios.
Documento 10 – API REST	Expõe os dados das entidades por meio de DTOs e endpoints.
Documento 12 – Banco de Dados PostgreSQL	Converte este modelo conceitual em estrutura física de tabelas, índices e restrições.

3.25 Conclusão do Capítulo

As entidades apresentadas representam o núcleo do modelo de dados do GTP e traduzem os conceitos fundamentais do domínio da aplicação. A organização por módulos, a padronização dos atributos comuns e a definição clara de responsabilidades proporcionam uma base sólida para a implementação da persistência, assegurando consistência, integridade e escalabilidade.

Nos capítulos seguintes, essas entidades serão conectadas por meio dos relacionamentos e cardinalidades, culminando na construção do Diagrama Entidade-Relacionamento (DER) completo, que servirá como referência para o banco de dados PostgreSQL e para o mapeamento JPA/Hibernate.

4 – Relacionamentos e Cardinalidades

4.1 Objetivo

Este capítulo define os relacionamentos existentes entre as entidades do GTP, especificando:

- tipo do relacionamento;
- cardinalidade;
- entidade proprietária da relação;
- chave estrangeira correspondente;
- regras de integridade;
- comportamento esperado durante operações de inserção, atualização e exclusão.

Essas definições garantem consistência entre o modelo conceitual, o banco de dados PostgreSQL e a implementação em JPA/Hibernate.

4.2 Tipos de Relacionamento

O modelo utilizará quatro tipos fundamentais.

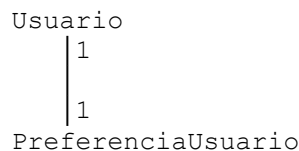
Um para Um (1:1)

Cada registro de uma entidade está associado a um único registro da outra entidade.

Exemplos:

- Usuário ↔ Preferências do Usuário
- Usuário ↔ Configuração Individual (caso implementada)

Representação



Implementação JPA

```

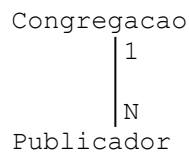
@OneToOne
@JoinColumn(name = "usuario_id")
private PreferenciaUsuario preferencia;
  
```

Um para Muitos (1:N)

É o relacionamento predominante no GTP.

Uma entidade "pai" pode possuir diversos registros relacionados.

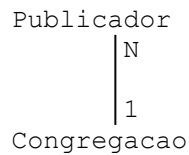
Exemplo:



Muitos para Um (N:1)

Representa o lado inverso do relacionamento anterior.

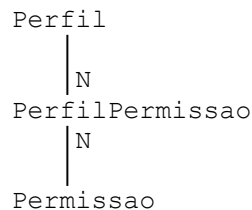
Exemplo:



Muitos para Muitos (N:N)

Sempre será implementado utilizando uma entidade associativa.

Exemplo:



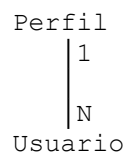
Essa abordagem facilita auditoria, inclusão de atributos adicionais e manutenção.

4.3 Relacionamentos do Domínio de Segurança

Usuário → Perfil

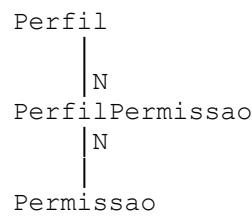
Item	Valor
Tipo	Muitos para Um (N:1)
FK	perfil_id
Obrigatório	Sim

Cardinalidade



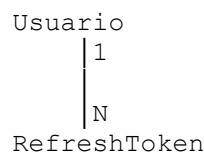
Perfil → Permissão

Item	Valor
Tipo	Muitos para Muitos
Tabela Associativa	perfil_permissao



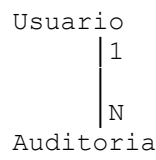
Usuário → Refresh Token

Um usuário poderá possuir diversos tokens válidos.



Usuário → Auditoria

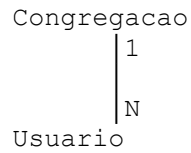
Todas as operações executadas por um usuário poderão gerar registros de auditoria.



4.4 Relacionamentos do Domínio Congregação

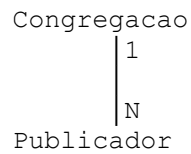
Congregação → Usuário

Uma congregação poderá possuir diversos usuários administrativos.

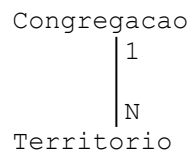


Congregação → Publicador

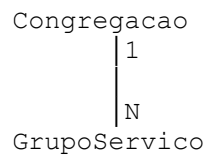
Relacionamento principal do sistema.



Congregação → Território



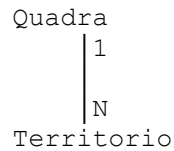
Congregação → Grupo de Serviço



4.5 Relacionamentos do Domínio Territórios

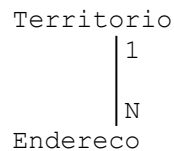
Quadra → Território

Uma quadra poderá conter diversos territórios.



Território → Endereço

Relacionamento fundamental do sistema.



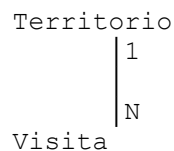
Cada endereço pertence obrigatoriamente a um território.

Território → Designação



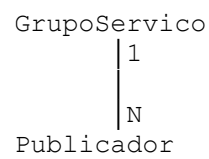
Mantém todo o histórico de utilização do território.

Território → Visita

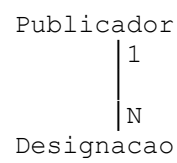


4.6 Relacionamentos do Domínio Publicadores

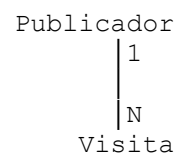
Grupo de Serviço → Publicador



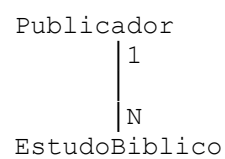
Publicador → Designação



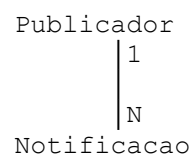
Publicador → Visita



Publicador → Estudo Bíblico

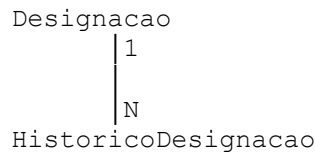


Publicador → Notificação



4.7 Relacionamentos do Domínio Designações

Designação → Histórico

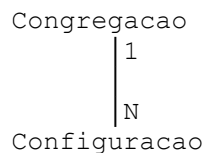


Cada alteração relevante gera um novo registro histórico.

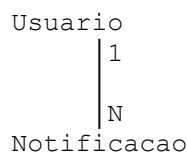
4.8 Relacionamentos Administrativos

Configuração → Congregação

Uma congregação poderá possuir diversas configurações específicas.



Usuário → Notificação



4.9 Entidades Associativas

As relações N:N serão implementadas por tabelas intermediárias.

Relação	Entidade Associativa
Perfil × Permissão	perfil_permissao
Publicador × Grupo Especial (caso implementado)	publicador_grupo
Usuário × Função Temporária (extensível)	usuario_funcao

Essas entidades poderão armazenar atributos adicionais, como datas de vigência, status e observações.

4.10 Cardinalidades Gerais

Origem	Destino	Cardinalidade
Perfil	Usuário	1:N
Usuário	RefreshToken	1:N
Usuário	Auditoria	1:N
Congregação	Usuário	1:N
Congregação	Publicador	1:N
Congregação	Território	1:N
Congregação	Grupo de Serviço	1:N
Quadra	Território	1:N
Território	Endereço	1:N
Território	Designação	1:N
Território	Visita	1:N
Grupo de Serviço	Publicador	1:N
Publicador	Designação	1:N
Publicador	Visita	1:N
Publicador	Estudo Bíblico	1:N
Publicador	Notificação	1:N
Designação	Histórico	1:N
Perfil	Permissão	N:N

4.11 Integridade Referencial

Todos os relacionamentos deverão utilizar chaves estrangeiras (FOREIGN KEY) e respeitar as seguintes diretrizes:

- não permitir referências inexistentes;
- impedir registros órfãos;
- garantir consistência entre tabelas;
- preservar o histórico quando aplicável.

As ações de exclusão e atualização deverão ser definidas conforme as regras de negócio:

Operação	Diretriz
ON DELETE RESTRICT	Entidades críticas, como Congregação e Perfil.
ON DELETE CASCADE	Tabelas associativas e dependentes sem valor histórico.
ON DELETE SET NULL	Relacionamentos opcionais, quando permitido.
ON UPDATE CASCADE	Quando necessário para manter consistência de chaves.

4.12 Mapeamento para JPA/Hibernate

Os relacionamentos serão implementados utilizando as anotações padrão da Jakarta Persistence.

Cardinalidade	Anotação
1:1	@OneToOne
1:N	@OneToMany
N:1	@ManyToOne
N:N	@ManyToMany (ou entidade associativa com @OneToMany/@ManyToOne)

Será dada preferência ao uso de **entidades associativas explícitas** em relacionamentos N:N, pois oferecem maior flexibilidade para evolução do domínio.

4.13 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a organização dos módulos que originam os relacionamentos.
Documento 06 – Arquitetura Backend	Implementa os relacionamentos nas entidades JPA.
Documento 10 – API REST	Expõe os relacionamentos por meio de DTOs e endpoints.
Documento 12 – Banco de Dados PostgreSQL	Implementa fisicamente as chaves estrangeiras, índices e restrições.

4.14 Conclusão do Capítulo

Os relacionamentos e cardinalidades definidos neste capítulo estruturam a interação entre as entidades do GTP e asseguram a integridade do modelo de dados. A padronização das cardinalidades, o uso consistente de chaves estrangeiras e a preferência por entidades associativas em relacionamentos complexos tornam o modelo mais robusto, flexível e alinhado às boas práticas de modelagem relacional e implementação com JPA/Hibernate.

5 – Integridade Referencial e Regras de Banco

5.1 Objetivo

Este capítulo estabelece as regras de integridade aplicáveis ao banco de dados do GTP.

São definidos:

- tipos de restrições;
- regras de integridade referencial;
- políticas de atualização e exclusão;
- validações de domínio;
- mecanismos de auditoria;
- controle de concorrência;
- diretrizes para evolução do esquema de banco.

Essas definições asseguram que os dados permaneçam consistentes durante todo o ciclo de vida da aplicação.

5.2 Integridade de Entidade

Toda tabela deverá possuir uma chave primária (PRIMARY KEY) responsável por identificar unicamente cada registro.

Diretrizes

- nome padronizado: id;
- tipo BIGINT;
- geração automática pelo PostgreSQL utilizando GENERATED BY DEFAULT AS IDENTITY;
- valor imutável após a criação.

Exemplo:

```
CREATE TABLE usuario (  
    id BIGINT GENERATED BY DEFAULT AS IDENTITY  
    PRIMARY KEY  
);
```

5.3 Integridade Referencial

Os relacionamentos entre entidades deverão ser protegidos por **chaves estrangeiras (FOREIGN KEY)**, impedindo referências inválidas.

Objetivos

- evitar registros órfãos;
- garantir consistência entre tabelas;
- preservar o histórico de informações;
- impedir exclusões que comprometam o domínio.

Exemplo:

```
ALTER TABLE publicador
ADD CONSTRAINT fk_publicador_congregacao
FOREIGN KEY (congregacao_id)
REFERENCES congregacao(id);
```

5.4 Restrições de Unicidade

Campos que representam identificadores naturais deverão possuir restrições UNIQUE.

Exemplos

Entidade	Campo
Usuário	email
Congregação	codigo
Território	numero
Perfil	nome
Permissão	codigo

Essas restrições evitam duplicidade de registros e reforçam a integridade do domínio.

5.5 Restrições CHECK

Sempre que possível, serão utilizadas restrições CHECK para validar valores diretamente no banco.

Exemplos

- status válidos;
- valores monetários não negativos;
- datas coerentes;
- campos numéricos dentro de limites definidos.

Exemplo:

```
CHECK (quantidade >= 0)
```

Outro exemplo:

```
CHECK (status IN ('ATIVO','INATIVO'))
```

Essas validações complementam as verificações realizadas pela aplicação.

5.6 Política de Exclusão

A exclusão física de registros será evitada sempre que possível.

Será adotada preferencialmente a estratégia de **soft delete**.

Campo padrão

ativo BOOLEAN

Valores:

Valor Significado

true Registro ativo

false Registro inativo

Vantagens

- preservação do histórico;
- recuperação de registros;
- suporte à auditoria;
- manutenção da integridade referencial.

A exclusão física será reservada para dados temporários, registros auxiliares ou situações previstas em requisitos legais.

5.7 Políticas ON DELETE e ON UPDATE

As ações associadas às chaves estrangeiras seguirão critérios definidos conforme o domínio.

Situação	Política Recomendada
Entidades críticas	ON DELETE RESTRICT
Tabelas associativas	ON DELETE CASCADE
Relacionamentos opcionais	ON DELETE SET NULL

Situação	Política Recomendada
Alteração de chaves	ON UPDATE CASCADE (quando aplicável)
Exemplos	
RESTRICT	
Impedir exclusão de uma congregação que ainda possua publicadores vinculados.	
CASCADE	
Excluir automaticamente registros de uma tabela associativa quando o registro principal for removido.	
SET NULL	
Remover a referência quando o relacionamento for opcional, preservando o registro.	

5.8 Integridade de Domínio

Os valores armazenados deverão respeitar as regras de negócio definidas no Documento 03.

Exemplos:

- um território deve pertencer a uma congregação;
- um publicador ativo deve estar vinculado a uma congregação;
- uma designação deve referenciar um território existente;
- um estudo bíblico deve possuir um publicador responsável.

Essas regras serão implementadas tanto na aplicação quanto, sempre que possível, no banco de dados.

5.9 Integridade Temporal

Campos de data deverão seguir regras cronológicas.

Exemplos:

- data de devolução não pode ser anterior à data de designação;
- data de atualização não pode ser anterior à data de criação;
- data de expiração de um token deve ser posterior à data de emissão.

Essas validações podem ser reforçadas por CHECK, quando aplicável, ou pela camada de aplicação.

5.10 Auditoria dos Dados

As principais entidades deverão registrar informações de auditoria.

Campos comuns:

Campo	Finalidade
data_criacao	Data da criação
criado_por	Usuário responsável
data_atualizacao	Última alteração
atualizado_por	Último usuário que alterou
versao	Controle de concorrência

Além disso, eventos relevantes serão registrados na tabela auditoria, conforme definido no Documento 08 – Segurança.

5.11 Controle de Concorrência

Será adotada a estratégia de **concorrência otimista** (*optimistic locking*).

Cada entidade persistente possuirá o campo:

versao BIGINT

Na implementação JPA, esse campo será anotado com @Version.

Benefícios:

- evita perda de atualizações;
- detecta conflitos entre usuários;
- melhora a escalabilidade em relação ao bloqueio pessimista.

5.12 Índices e Desempenho

Serão criados índices para otimizar consultas frequentes.

Campos recomendados

- e-mail;
- código da congregação;
- número do território;
- CPF (caso utilizado);
- data de designação;

- status;
- chaves estrangeiras.

Convenção

idx_nome_tabela_campo

Exemplo:

idx_usuario_email

idx_territorio_numero

idx_designacao_status

A criação de índices será equilibrada para não comprometer operações de escrita.

5.13 Validações na Aplicação

Além das restrições do banco, a aplicação deverá utilizar **Jakarta Validation**.

Principais anotações:

- @NotNull
- @NotBlank
- @Email
- @Size
- @Min
- @Max
- @Positive
- @Past
- @Future
- @Pattern

A validação em múltiplas camadas reduz a possibilidade de inconsistências.

5.14 Versionamento do Esquema

Todas as alterações estruturais deverão ser controladas por meio do **Flyway**.

Diretrizes:

- scripts imutáveis;
- numeração sequencial;
- descrição clara das migrações;
- rastreabilidade completa.

Exemplo de nomenclatura:

```
V1__criacao_tabela_usuario.sql  
V2__criacao_tabela_congregacao.sql  
V3__criacao_relacionamentos.sql
```

5.15 Recuperação e Consistência

As operações críticas deverão ser executadas em transações.

Diretrizes:

- atomicidade;
- consistência;
- isolamento;
- durabilidade (ACID).

Na camada de aplicação, será utilizado @Transactional para garantir a integridade das operações.

5.16 Relação com os Demais Documentos

Documento	Relação
Documento 03 – Regras de Negócio	Origina diversas restrições implementadas no banco.
Documento 06 – Arquitetura Backend	Implementa as validações e transações definidas neste capítulo.
Documento 08 – Segurança	Define auditoria e rastreabilidade dos dados.
Documento 09 – Docker	Disponibiliza o ambiente PostgreSQL para aplicação das migrações.
Documento 12 – Banco de Dados PostgreSQL	Concretiza fisicamente todas as restrições descritas neste capítulo.

5.17 Boas Práticas

Durante a implementação do banco de dados deverão ser observadas as seguintes recomendações:

- evitar redundância de informações;

- utilizar nomes padronizados para tabelas e colunas;
- definir todas as chaves estrangeiras;
- criar índices apenas quando justificados;
- documentar restrições especiais;
- utilizar transações para operações críticas;
- evitar exclusões físicas de entidades de negócio;
- manter sincronização entre o modelo conceitual, as entidades JPA e as migrações Flyway.

5.18 Conclusão do Capítulo

As regras de integridade apresentadas neste capítulo garantem que o banco de dados do GTP preserve a consistência das informações, minimize riscos de inconsistências e suporte a evolução contínua da plataforma. A combinação de chaves primárias, estrangeiras, restrições de unicidade, validações de domínio, auditoria, *soft delete* e controle de concorrência estabelece uma base sólida para a implementação em PostgreSQL e para o mapeamento objeto-relacional com JPA/Hibernate.

6 – Índices, Desempenho e Otimização

6.1 Objetivo

Este capítulo apresenta as estratégias de otimização do banco de dados do GTP.

São definidos:

- critérios para criação de índices;
- otimização de consultas;
- boas práticas de modelagem para desempenho;
- manutenção dos índices;
- utilização eficiente do PostgreSQL;
- integração com JPA/Hibernate.

O objetivo é garantir que o crescimento da base de dados não comprometa a experiência dos usuários nem a eficiência da aplicação.

6.2 Estratégia Geral de Indexação

A criação de índices deverá priorizar colunas utilizadas com frequência em:

- pesquisas (WHERE);
- ordenações (ORDER BY);
- agrupamentos (GROUP BY);
- junções (JOIN);
- filtros compostos;
- chaves estrangeiras.

Os índices deverão ser planejados de forma equilibrada, considerando que também impactam operações de escrita (INSERT, UPDATE e DELETE).

6.3 Índices Primários

Todas as tabelas possuirão índice automático sobre a chave primária.

Exemplo:

PRIMARY KEY (id)

Características:

- único;
- criado automaticamente pelo PostgreSQL;
- utilizado em pesquisas por identificador.

6.4 Índices em Chaves Estrangeiras

Todas as colunas utilizadas como FOREIGN KEY deverão possuir índices próprios.

Exemplos:

Tabela	Campo Indexado
usuario	perfil_id
usuario	congregacao_id
publicador	congregacao_id
territorio	congregacao_id
endereco	territorio_id
designacao	publicador_id

Tabela	Campo Indexado
--------	----------------

designacao	territorio_id
------------	---------------

auditoria	usuario_id
-----------	------------

Benefícios:

- aceleração de JOINS;
- melhoria em filtros por relacionamento;
- melhor desempenho na integridade referencial.

6.5 Índices Únicos

Campos que representam identificadores naturais deverão utilizar índices únicos.

Exemplos:

Campo	Tipo
usuario.email	UNIQUE
congregacao.codigo	UNIQUE
perfil.nome	UNIQUE
permissao.codigo	UNIQUE
territorio.numero	UNIQUE (por congregação, se aplicável)

Esses índices garantem integridade e consultas eficientes.

6.6 Índices Compostos

Consultas frequentes que utilizam mais de uma coluna deverão ser atendidas por índices compostos.

Exemplos:

Índice	Utilização
(congregacao_id, ativo)	Publicadores ativos por congregação
(territorio_id, status)	Designações por território
(usuario_id, data_criacao)	Auditoria por usuário
(publicador_id, data_designacao)	Histórico de designações

A ordem das colunas deverá refletir os padrões reais de consulta da aplicação.

6.7 Índices Parciais

Quando apenas parte dos registros é consultada com frequência, poderão ser utilizados índices parciais.

Exemplo:

```
CREATE INDEX idx_publicador_ativo
ON publicador (congregacao_id)
WHERE ativo = true;
```

Vantagens:

- menor consumo de armazenamento;
- melhor desempenho em consultas específicas;
- redução do custo de manutenção do índice.

6.8 Consultas Otimizadas

As consultas deverão:

- selecionar apenas as colunas necessárias;
- utilizar filtros indexados;
- evitar operações desnecessárias;
- limitar resultados quando apropriado.

Exemplo recomendado:

```
SELECT id, nome
FROM publicador
WHERE congregacao_id = ?
ORDER BY nome
LIMIT 20;

Evitar:

SELECT *
FROM publicador;
```

quando nem todas as colunas forem necessárias.

6.9 Paginação

Grandes volumes de dados deverão ser paginados.

No PostgreSQL:

```
LIMIT 20 OFFSET 40;
```

Na API REST:

```
GET /publicadores?page=2&size=20
```

Na camada JPA:

- Pageable;
- Page<T>;
- Slice<T> para consultas sem necessidade de contagem total.

A paginação reduz o consumo de memória e melhora o tempo de resposta.

6.10 Otimização de Relacionamentos

Para evitar consultas excessivas (*N+1 Query Problem*), deverão ser adotadas estratégias de carregamento adequadas.

Recomendações

- utilizar LAZY como padrão para coleções;
- aplicar JOIN FETCH quando necessário;
- utilizar projeções (DTOs) para consultas específicas;
- evitar EAGER indiscriminadamente.

Essas práticas reduzem a quantidade de consultas executadas e melhoram o desempenho geral.

6.11 Análise de Planos de Execução

Consultas críticas deverão ser analisadas periodicamente utilizando:

```
EXPLAIN
```

e

```
EXPLAIN ANALYZE
```

Essas ferramentas permitem identificar:

- uso inadequado de índices;
- varreduras completas (*Sequential Scan*);

- junções ineficientes;
- gargalos de desempenho.

Os resultados deverão orientar ajustes na modelagem e na indexação.

6.12 Estatísticas e Manutenção

O PostgreSQL utiliza estatísticas para otimizar a execução das consultas.

Recomenda-se a execução periódica de:

VACUUM

e

ANALYZE

Funções:

- remover espaço de registros obsoletos;
- atualizar estatísticas;
- melhorar os planos de execução.

Em ambientes de produção, o recurso autovacuum deverá permanecer habilitado.

6.13 Particionamento

Embora a versão inicial do GTP não exija particionamento, o modelo deverá permitir sua adoção futura para tabelas com grande volume de dados.

Tabelas candidatas:

- auditoria;
- historico_designacao;
- visita;
- notificacao.

Critérios de particionamento:

- por data;
- por congregação (casos específicos);
- por período de retenção.

Essa estratégia facilita manutenção e melhora o desempenho em grandes bases.

6.14 Cache

Para consultas frequentemente executadas e de baixa volatilidade, poderão ser utilizados mecanismos de cache.

Possíveis aplicações:

- configurações do sistema;
- perfis e permissões;
- parâmetros globais;
- listas de referência.

A estratégia de cache deverá garantir consistência com os dados persistidos.

6.15 Monitoramento de Desempenho

Os seguintes indicadores deverão ser monitorados:

Indicador	Objetivo
Tempo médio de consulta	Avaliar desempenho
Consultas mais lentas	Identificar gargalos
Uso de índices	Verificar eficiência
Leituras por tabela	Detectar padrões de acesso
Escritas por tabela	Avaliar carga operacional
Bloqueios	Identificar contenção
Crescimento das tabelas	Planejar capacidade

Essas métricas deverão ser acompanhadas em ambientes de homologação e produção.

6.16 Recomendações para JPA/Hibernate

A implementação deverá observar as seguintes boas práticas:

- utilizar Pageable para listagens;
- evitar consultas desnecessárias;
- preferir DTOs para projeções;
- utilizar EntityGraph quando apropriado;
- controlar o tamanho do contexto de persistência;
- evitar carregamento antecipado de coleções extensas;

- revisar consultas geradas automaticamente pelo Hibernate.

Essas práticas contribuem para uma camada de persistência mais eficiente.

6.17 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Implementa as estratégias de acesso aos dados.
Documento 09 – Docker	Define o ambiente PostgreSQL utilizado para testes de desempenho.
Documento 10 – API REST	Utiliza paginação, filtros e ordenação definidos neste capítulo.
Documento 12 – Banco de Dados PostgreSQL	Implementa fisicamente índices, estatísticas e estratégias de otimização.

6.18 Boas Práticas

Para garantir desempenho consistente ao longo da evolução do sistema, recomenda-se:

- criar índices apenas quando justificados;
- revisar periodicamente consultas críticas;
- evitar `SELECT *` em operações de produção;
- utilizar paginação em listagens extensas;
- monitorar planos de execução;
- manter estatísticas atualizadas;
- aplicar *lazy loading* por padrão;
- revisar índices conforme o crescimento da base de dados;
- realizar testes de carga antes de grandes implantações.

6.19 Conclusão do Capítulo

As estratégias de indexação, otimização e monitoramento apresentadas neste capítulo fornecem uma base sólida para que o banco de dados do GTP mantenha alto desempenho mesmo com o aumento do volume de informações e de usuários. A combinação de índices bem planejados, consultas eficientes, manutenção preventiva e

boas práticas no uso do PostgreSQL e do JPA/Hibernate contribuirá para uma plataforma escalável, estável e preparada para evolução contínua.

7 – Diagrama Entidade-Relacionamento (DER)

7.1 Objetivo

Este capítulo apresenta a representação estrutural do banco de dados do GTP por meio do Diagrama Entidade-Relacionamento (DER).

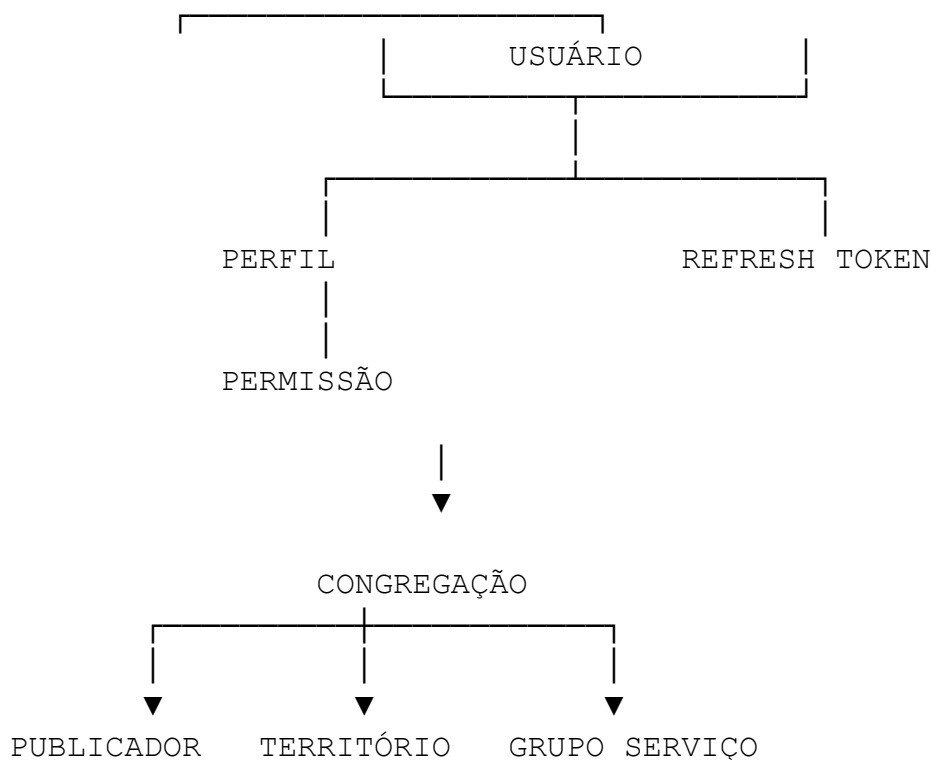
O objetivo é:

- representar graficamente as entidades do sistema;
- demonstrar os relacionamentos entre os domínios;
- identificar as cardinalidades;
- servir como base para implementação física no PostgreSQL;
- orientar o mapeamento JPA/Hibernate.

O diagrama reflete a modelagem conceitual e lógica definida nos capítulos anteriores.

7.2 Visão Geral do Modelo

O modelo de dados do GTP está organizado em módulos funcionais independentes, porém integrados.

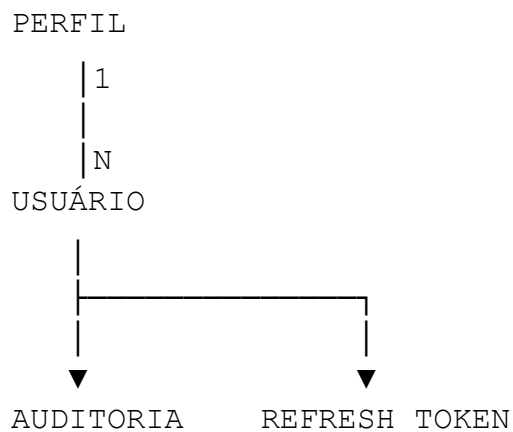




Essa organização evidencia a separação por domínios e reduz o acoplamento entre módulos.

7.3 Domínio de Segurança

O domínio de segurança concentra os elementos responsáveis pela autenticação e autorização.



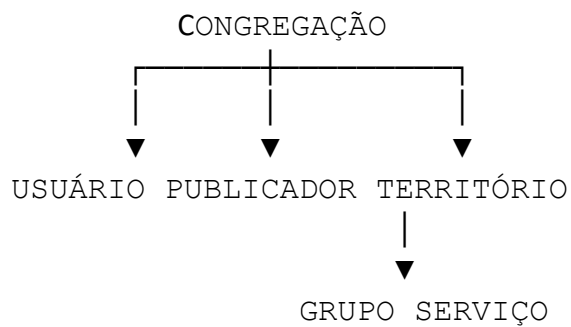


Principais relacionamentos

Origem	Destino	Cardinalidade
Perfil	Usuário	1:N
Perfil	Permissão	N:N
Usuário	Auditoria	1:N
Usuário	Refresh Token	1:N

7.4 Domínio Congregação

A congregação representa o núcleo organizacional do sistema.



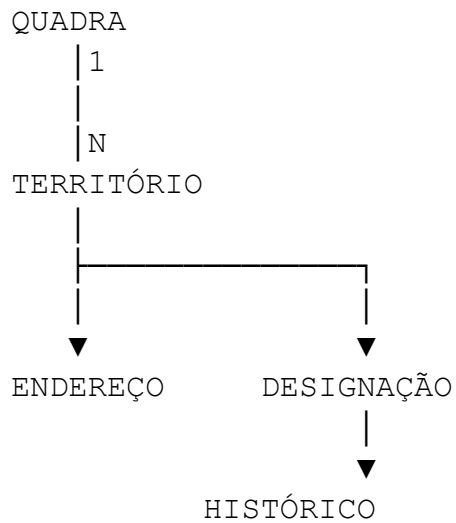
Relacionamentos principais

- Congregação → Usuários
- Congregação → Publicadores
- Congregação → Territórios
- Congregação → Grupos de Serviço

Todos:

1:N

7.5 Domínio Territórios



Relacionamentos

Origem Destino

Quadra Território

Território Endereço

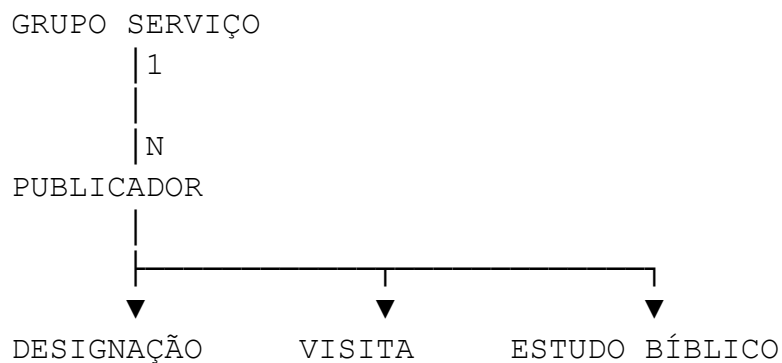
Território Designação

Designação Histórico

Todos:

1:N

7.6 Domínio Publicadores



Relacionamentos

- Grupo → Publicadores
- Publicador → Designações
- Publicador → Visitas

- Publicador → Estudos

Todos:

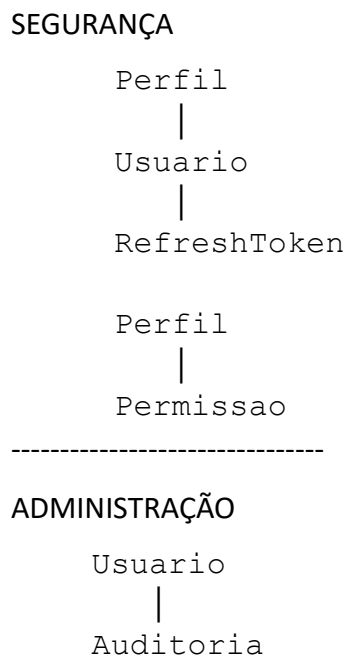
1:N

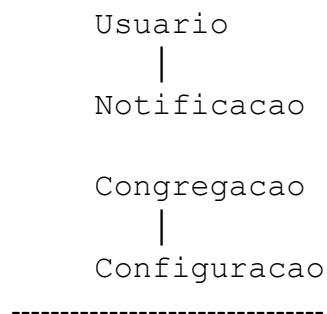
7.7 Domínio Administrativo



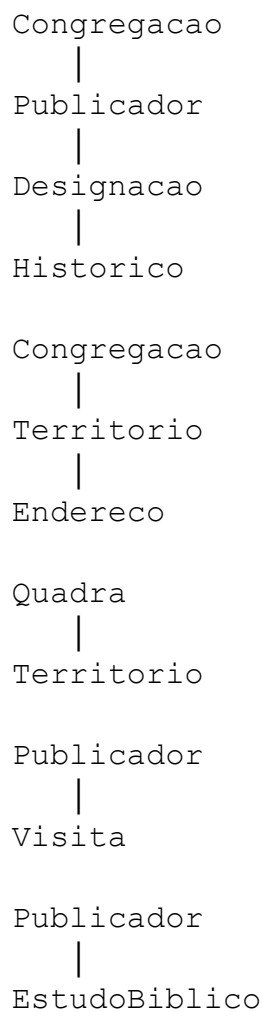
7.8 Modelo Lógico Consolidado

A estrutura lógica geral do banco pode ser resumida da seguinte forma:





NEGÓCIO



Essa organização favorece a modularização e facilita a implementação por domínios no backend.

7.9 Resumo das Cardinalidades

Relacionamento	Cardinalidade
Perfil → Usuário	1:N

Relacionamento	Cardinalidade
Perfil → Permissão	N:N
Usuário → Auditoria	1:N
Usuário → Refresh Token	1:N
Congregação → Usuário	1:N
Congregação → Publicador	1:N
Congregação → Território	1:N
Congregação → Grupo de Serviço	1:N
Quadra → Território	1:N
Território → Endereço	1:N
Território → Designação	1:N
Designação → Histórico	1:N
Publicador → Designação	1:N
Publicador → Visita	1:N
Publicador → Estudo Bíblico	1:N
Usuário → Notificação	1:N

7.10 Entidades Associativas

Os relacionamentos do tipo N:N serão implementados por tabelas intermediárias.

Relação	Entidade Associativa
Perfil × Permissão	perfil_permissao
Publicador × Grupo Especial (extensível)	publicador_grupo
Usuário × Função Temporária (extensível)	usuario_funcao

Essa abordagem permite a inclusão de atributos adicionais, como datas de vigência, status e observações.

7.11 Correspondência com JPA/Hibernate

Os relacionamentos do DER serão mapeados utilizando as anotações padrão da Jakarta Persistence.

Cardinalidade Anotação

1:1	@OneToOne
1:N	@OneToMany
N:1	@ManyToOne
N:N	@ManyToMany ou entidade associativa com @OneToMany e @ManyToOne

Será priorizado o uso de entidades associativas explícitas para relacionamentos N:N, favorecendo a extensibilidade do domínio.

7.12 Evolução do Modelo

O DER foi concebido para permitir a incorporação de novos módulos sem necessidade de alterações estruturais significativas.

Exemplos de futuras expansões:

- integração com mapas geográficos (GIS);
- roteirização inteligente de territórios;
- sincronização com aplicativos móveis;
- gestão documental;
- integração com calendários;
- estatísticas avançadas;
- módulos de importação e exportação de dados.

Novas entidades deverão seguir os padrões definidos neste documento.

7.13 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define os módulos representados no DER.
Documento 06 – Arquitetura Backend	Implementa o modelo por meio de entidades JPA/Hibernate.
Documento 10 – API REST	Expõe os dados modelados neste diagrama através dos endpoints.

Documento	Relação
Documento 12 – Banco de Dados PostgreSQL	Traduz o DER em tabelas, índices, restrições e migrações Flyway.

7.14 Boas Práticas para Manutenção do DER

Para garantir que o diagrama permaneça consistente durante a evolução do sistema, recomenda-se:

- manter o DER sincronizado com as migrações Flyway;
- revisar os relacionamentos antes da criação de novas entidades;
- evitar duplicidade de informações entre módulos;
- documentar alterações estruturais no histórico do projeto;
- validar o impacto de mudanças nas entidades JPA e nos contratos da API;
- utilizar ferramentas de modelagem para manter uma versão gráfica atualizada do diagrama.

7.15 Conclusão do Capítulo

O Diagrama Entidade-Relacionamento consolida toda a modelagem de dados do GTP em uma visão unificada, demonstrando as entidades, seus relacionamentos e as cardinalidades que sustentam o domínio da aplicação. Ele servirá como referência central para a implementação física no PostgreSQL, para o mapeamento objeto-relacional com JPA/Hibernate e para a evolução futura da plataforma.

8 – Mapeamento para JPA/Hibernate

8.1 Objetivo

Este capítulo define as diretrizes para o mapeamento das entidades do Modelo Entidade-Relacionamento (DER) para objetos Java utilizando Jakarta Persistence (JPA) e Hibernate.

São abordados:

- organização das entidades;
- herança e reutilização;
- mapeamento de atributos;
- relacionamentos;

- estratégias de carregamento;
- cascatas;
- auditoria;
- concorrência otimista;
- boas práticas para Spring Data JPA.

O objetivo é assegurar que a camada de persistência reflita fielmente o modelo de domínio e mantenha alto desempenho, integridade e facilidade de manutenção.

8.2 Estrutura do Pacote de Entidades

As entidades serão organizadas por domínio funcional, acompanhando a arquitetura modular do projeto.

```
src/main/java
├── br/com/gtp
│   ├── security
│   │   └── entity
│   ├── congregation
│   │   └── entity
│   ├── territory
│   │   └── entity
│   ├── publisher
│   │   └── entity
│   ├── assignment
│   │   └── entity
│   ├── visit
│   │   └── entity
│   ├── study
│   │   └── entity
│   ├── notification
│   │   └── entity
│   ├── audit
│   │   └── entity
│   └── common
│       └── Entity
```

Essa organização facilita a manutenção e reduz o acoplamento entre módulos.

8.3 Classe Base (BaseEntity)

As entidades compartilharão uma superclasse abstrata contendo atributos comuns.

Campos compartilhados

Campo	Finalidade
<code>id</code>	Identificador único
<code>ativo</code>	Exclusão lógica
<code>dataCriacao</code>	Auditoria
<code>dataAtualizacao</code>	Auditoria
<code>criadoPor</code>	Usuário criador
<code>atualizadoPor</code>	Usuário responsável pela alteração
<code>versao</code>	Controle de concorrência otimista

A classe será anotada com:

- `@MappedSuperclass`
- `@EntityListeners`
- `@Version` (campo `versao`)

Essa abordagem evita duplicação de código e padroniza a persistência.

8.4 Identificadores

Todas as entidades utilizarão:

- `@Id;`
- `@GeneratedValue(strategy = GenerationType.IDENTITY)`.

Exemplo conceitual:

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

Essa estratégia é compatível com o PostgreSQL e simplifica a geração de chaves primárias.

8.5 Mapeamento de Atributos

Os atributos serão mapeados utilizando `@Column`.

Diretrizes

- nomes em `snake_case`;

- comprimento definido para campos VARCHAR;
- obrigatoriedade refletida por nullable = false;
- unicidade representada por unique = true, quando aplicável.

Exemplo:

```
@Column(name = "email", nullable = false, unique =
true, length = 150)
private String email;
```

8.6 Relacionamentos

Os relacionamentos seguirão diretamente o DER definido no Capítulo 7.

Um para Um

Utilizar:

- @OneToOne
- @JoinColumn

Um para Muitos

Utilizar:

- @OneToMany(mappedBy = "...")

Muitos para Um

Utilizar:

- @ManyToOne
- @JoinColumn

Muitos para Muitos

Preferencialmente implementado por entidades associativas.

Quando necessário:

- @ManyToMany
- @JoinTable

Essa abordagem proporciona maior flexibilidade para evolução do domínio.

8.7 Estratégias de Carregamento

Para otimizar o desempenho, serão adotadas estratégias adequadas de carregamento das associações.

LAZY

Será o padrão para coleções e relacionamentos que não precisam ser carregados imediatamente.

Vantagens:

- menor consumo de memória;
- redução do número de consultas;
- melhor desempenho em operações de listagem.

EAGER

Será utilizado apenas quando o carregamento imediato for essencial para a regra de negócio.

O uso indiscriminado de EAGER deverá ser evitado para prevenir o problema conhecido como *N+1 Query*.

8.8 Cascatas

As operações em entidades relacionadas deverão utilizar CascadeType apenas quando houver dependência de ciclo de vida.

Principais estratégias:

Tipo	Utilização
PERSIST	Persistência conjunta
MERGE	Atualização conjunta
REMOVE	Exclusão de dependentes
REFRESH	Atualização dos dados persistidos
DETACH	Remoção do contexto de persistência
ALL	Utilizado apenas quando apropriado

O uso de CascadeType.ALL será restrito a entidades fortemente dependentes.

8.9 Exclusão Lógica (*Soft Delete*)

As entidades de negócio utilizarão exclusão lógica.

Campo padrão:

ativo BOOLEAN

A aplicação deverá considerar apenas registros ativos nas consultas padrão.

A exclusão física será reservada para dados temporários ou requisitos específicos.

8.10 Auditoria

As entidades deverão registrar automaticamente:

- data de criação;
- usuário criador;
- data de atualização;
- usuário responsável pela alteração.

A auditoria será integrada ao Spring Data JPA por meio de `AuditingEntityListener`.

Essa abordagem elimina a necessidade de preenchimento manual desses campos.

8.11 Controle de Concorrência

Será adotada concorrência otimista.

Campo:

```
versao
```

Mapeado com:

```
@Version  
private Long versao;
```

Benefícios:

- prevenção de sobrescritas simultâneas;
- detecção de conflitos de atualização;
- maior escalabilidade em ambientes com múltiplos usuários.

8.12 Validações

As entidades utilizarão **Jakarta Validation**.

Principais anotações:

Anotação	Finalidade
-----------------	-------------------

<code>@NotNull</code>	Campo obrigatório
-----------------------	-------------------

<code>@NotBlank</code>	Texto obrigatório
------------------------	-------------------

<code>@Email</code>	Validação de e-mail
---------------------	---------------------

Anotação	Finalidade
@Size	Tamanho mínimo/máximo
@Pattern	Expressões regulares
@Positive	Valores positivos
@Past	Datas passadas
@Future	Datas futuras

Essas validações complementam as restrições do banco de dados.

8.13 Enumerações

Valores controlados serão representados por enum.

Exemplos:

- StatusTerritorio
- StatusDesignacao
- TipoPublicacao
- Sexo
- TipoNotificacao

O armazenamento preferencial será por nome (EnumType.STRING), garantindo maior legibilidade e evitando problemas decorrentes da alteração da ordem dos valores.

8.14 Repositórios

Cada entidade persistente possuirá um repositório específico baseado em JpaRepository.

Exemplo de responsabilidades:

- operações CRUD;
- consultas por critérios;
- paginação;
- ordenação;
- consultas personalizadas.

Consultas complexas deverão utilizar projeções, Specification ou QueryDSL (quando adotado).

8.15 Consultas e Performance

Para manter a eficiência da camada de persistência, deverão ser observadas as seguintes práticas:

- utilizar paginação (Pageable) em listagens;
- evitar carregamento desnecessário de relacionamentos;
- utilizar JOIN FETCH quando necessário;
- preferir DTOs para consultas específicas;
- revisar consultas geradas automaticamente pelo Hibernate.

8.16 Transações

Operações que alteram o estado do banco deverão ser executadas dentro de transações.

Diretrizes:

- utilizar `@Transactional` na camada de serviço;
- manter transações curtas;
- evitar chamadas externas dentro de transações;
- garantir atomicidade das operações críticas.

8.17 Integração com Flyway

O mapeamento JPA deverá permanecer sincronizado com as migrações do Flyway.

Fluxo recomendado:

1. atualização do DER;
2. criação da migração Flyway;
3. atualização das entidades JPA;
4. implementação dos repositórios;
5. validação em ambiente de desenvolvimento.

Essa sequência reduz inconsistências entre o código e o banco de dados.

8.18 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Define a organização da camada de persistência.
Documento 08 – Segurança	Estabelece requisitos de auditoria implementados nas entidades.
Documento 10 – API REST	Consome as entidades por meio de DTOs e serviços.
Documento 12 – Banco de Dados PostgreSQL	Implementa fisicamente as tabelas e restrições correspondentes às entidades JPA.

8.19 Boas Práticas

Durante a implementação deverão ser observadas as seguintes recomendações:

- utilizar uma BaseEntity para atributos comuns;
- preferir carregamento LAZY;
- limitar o uso de CascadeType.ALL;
- mapear relacionamentos de forma explícita;
- evitar lógica de negócio nas entidades;
- manter sincronização entre DER, entidades JPA e migrações Flyway;
- utilizar validações em múltiplas camadas (aplicação e banco);
- revisar periodicamente o desempenho das consultas geradas pelo Hibernate.

8.20 Conclusão do Capítulo

O mapeamento para JPA/Hibernate transforma o modelo conceitual do GTP em uma camada de persistência robusta e alinhada às boas práticas do ecossistema Spring. A utilização de uma classe base compartilhada, relacionamentos bem definidos, estratégias adequadas de carregamento, validações consistentes e controle de concorrência garante uma implementação preparada para evolução, manutenção e operação em ambientes de produção.

9 – Evolução do Modelo e Migrações

9.1 Objetivo

Este capítulo define a estratégia para evolução do modelo de dados do Gestor de Territórios e Publicações (GTP), estabelecendo regras para criação, alteração e manutenção do esquema do banco de dados ao longo do ciclo de vida da aplicação.

São abordados:

- versionamento do banco de dados;
- utilização do Flyway;
- evolução incremental do modelo;
- compatibilidade entre versões;
- governança das alterações estruturais;
- boas práticas para migrações.

Essas diretrizes garantem que o banco de dados evolua de forma previsível, auditável e consistente entre todos os ambientes.

9.2 Princípios da Evolução do Modelo

A evolução do esquema do banco seguirá os seguintes princípios:

- versionamento obrigatório de todas as alterações;
- migrações incrementais e imutáveis;
- compatibilidade entre ambientes;
- rastreabilidade completa;
- automatização da aplicação das migrações;
- preservação da integridade dos dados.

Nenhuma alteração estrutural deverá ser realizada diretamente no banco de dados em produção.

9.3 Utilização do Flyway

O Flyway será a ferramenta oficial para controle de versão do esquema do banco de dados.

Suas responsabilidades incluem:

- criação de tabelas;

- inclusão de colunas;
- alteração de restrições;
- criação de índices;
- criação de *views*;
- atualização de dados estruturais;
- controle do histórico de migrações.

Todas as migrações serão executadas automaticamente durante a inicialização da aplicação.

9.4 Estrutura das Migrações

Os scripts serão organizados em diretório específico.

```
src/
├── main/
│   └── resources/
│       └── db/
│           └── migration/
│               ├── V1__criacao_usuario.sql
│               ├── V2__criacao_congregacao.sql
│               ├── V3__criacao_territorio.sql
│               ├── V4__criacao_publicador.sql
│               ├── V5__relacionamentos.sql
│               └── ...
```

Cada arquivo representará uma única alteração lógica no esquema do banco.

9.5 Convenção de Nomenclatura

As migrações seguirão o padrão:

V<versão>__<descrição>.sql

Exemplos:

```
V1__criacao_tabela_usuario.sql
V2__criacao_tabela_perfil.sql
V3__criacao_tabela_congregacao.sql
V4__criacao_relacionamentos.sql
V5__criacao_indices.sql
```

Diretrizes:

- numeração sequencial;

- descrição clara e objetiva;
- uso de letras minúsculas e *snake_case*;
- evitar abreviações pouco compreensíveis.

9.6 Versionamento Incremental

Cada modificação estrutural deverá resultar em uma nova versão do esquema.

Exemplos de alterações que exigem nova migração:

- criação de tabela;
- inclusão de coluna;
- alteração de tipo de dado;
- criação de índice;
- inclusão de restrição;
- alteração de relacionamento;
- criação de *view*;
- inserção de dados de referência.

Scripts já executados não deverão ser modificados.

9.7 Compatibilidade entre Versões

A evolução do banco deverá preservar a compatibilidade entre diferentes versões da aplicação sempre que possível.

Recomendações:

- evitar remoção imediata de colunas utilizadas;
- realizar alterações em etapas;
- manter versões intermediárias compatíveis durante implantações graduais;
- comunicar alterações que exijam adaptação da aplicação.

Essa estratégia reduz riscos durante atualizações em produção.

9.8 Migrações de Dados

Além de alterações estruturais, poderão existir migrações destinadas à transformação ou inicialização de dados.

Exemplos:

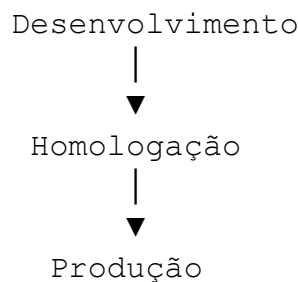
- inclusão de perfis padrão;
- cadastro de permissões iniciais;
- parâmetros do sistema;
- configurações globais;
- dados de referência.

Essas migrações deverão ser idempotentes sempre que possível, evitando efeitos indesejados em reexecuções controladas.

9.9 Ambientes

As migrações deverão ser aplicadas de forma consistente em todos os ambientes.

Fluxo recomendado:



A ordem e o conteúdo das migrações deverão permanecer idênticos entre os ambientes.

9.10 Controle de Qualidade

Antes da liberação de uma migração, recomenda-se validar:

- sintaxe SQL;
- impacto sobre dados existentes;
- tempo estimado de execução;
- compatibilidade com a versão atual da aplicação;
- reversibilidade (quando aplicável);
- impacto em índices e desempenho.

Essas verificações reduzem riscos durante implantações.

9.11 Estratégia para Alterações Complexas

Alterações de grande impacto deverão ser realizadas em etapas.

Fluxo recomendado:

1. adicionar novas estruturas;
2. migrar os dados;
3. adaptar a aplicação;
4. validar funcionamento;
5. remover estruturas antigas em migração posterior.

Essa abordagem minimiza indisponibilidade e facilita a reversão de mudanças.

9.12 Governança das Alterações

Toda alteração estrutural deverá ser registrada e aprovada conforme o fluxo de desenvolvimento do projeto.

Cada migração deverá possuir:

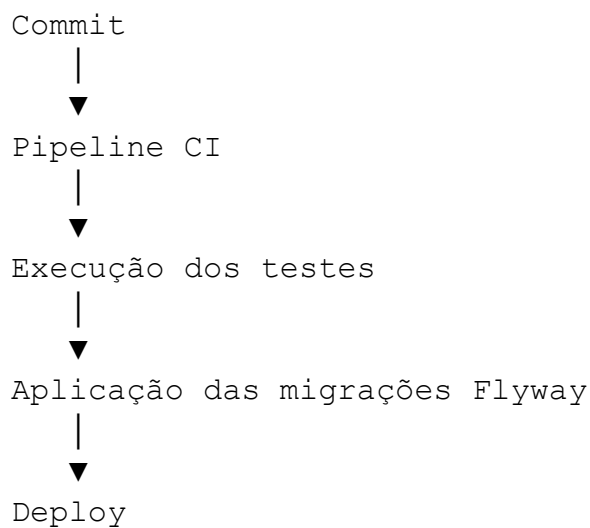
- identificação da versão;
- descrição da alteração;
- data de criação;
- responsável pela implementação;
- referência ao requisito ou caso de uso relacionado.

Essa documentação facilita auditorias e manutenção.

9.13 Integração com CI/CD

As migrações serão executadas automaticamente durante os processos de integração e implantação contínuas.

Fluxo previsto:



Caso uma migração falhe, a implantação deverá ser interrompida para evitar inconsistências.

9.14 Monitoramento das Migrações

Após a execução das migrações, deverão ser monitorados:

- tempo de execução;
- sucesso ou falha;
- consistência do esquema;
- integridade dos dados;
- utilização de índices;
- impacto sobre consultas críticas.

Essas informações auxiliam na identificação precoce de problemas.

9.15 Evolução do Modelo de Dados

O modelo foi concebido para permitir a inclusão de novos módulos sem necessidade de reestruturações significativas.

Possíveis evoluções futuras:

- georreferenciamento de territórios;
- integração com serviços de mapas;
- aplicativo móvel;

- sincronização offline;
- assinatura eletrônica de documentos;
- notificações em tempo real;
- geração avançada de indicadores;
- integração com serviços externos.

Novas entidades deverão seguir as convenções estabelecidas neste documento.

9.16 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a organização geral da solução.
Documento 06 – Arquitetura Backend	Implementa as entidades e repositórios afetados pelas migrações.
Documento 09 – Docker	Disponibiliza o ambiente PostgreSQL utilizado para execução do Flyway.
Documento 10 – API REST	Consome a estrutura evoluída do banco por meio dos serviços e DTOs.
Documento 12 – Banco de Dados PostgreSQL	Detalha a implementação física das tabelas, índices, restrições e scripts SQL.

9.17 Boas Práticas

Para garantir uma evolução segura do banco de dados, recomenda-se:

- nunca alterar migrações já executadas;
- criar uma migração para cada alteração lógica;
- manter descrições claras e padronizadas;
- validar migrações em ambiente de desenvolvimento antes da homologação;
- evitar alterações destrutivas em uma única etapa;
- sincronizar o DER, as entidades JPA e as migrações Flyway;
- documentar todas as alterações estruturais;
- monitorar o desempenho após grandes mudanças no esquema.

9.18 Conclusão do Capítulo

A estratégia de evolução e migrações apresentada neste capítulo assegura que o banco de dados do GTP possa acompanhar o crescimento funcional da aplicação de forma controlada e segura. O uso do Flyway, aliado a práticas de versionamento incremental, governança das alterações e integração com os processos de CI/CD, proporciona rastreabilidade, previsibilidade e estabilidade ao longo de todo o ciclo de vida do sistema.

Essas diretrizes permitem que novas funcionalidades sejam incorporadas sem comprometer a integridade dos dados ou a continuidade operacional da plataforma.

10 – Conclusão

10.1 Objetivo da Modelagem

O Modelo Entidade-Relacionamento (DER) apresentado neste documento estabelece a estrutura lógica e conceitual que sustentará toda a camada de persistência do **Gestor de Territórios e Publicações (GTP)**.

A modelagem foi desenvolvida com o propósito de representar fielmente os conceitos do domínio da aplicação, organizando entidades, atributos, relacionamentos e restrições de forma consistente, escalável e aderente às melhores práticas de engenharia de software.

Ao longo deste documento foram definidos os princípios de modelagem, as entidades do sistema, suas cardinalidades, regras de integridade e diretrizes de implementação, formando uma base sólida para o desenvolvimento do banco de dados PostgreSQL e da camada de persistência em Spring Boot.

10.2 Objetivos Alcançados

Com a elaboração deste documento foram atingidos os seguintes objetivos:

- definição das entidades que representam o domínio do GTP;
- identificação dos atributos e respectivas responsabilidades;
- padronização das chaves primárias e estrangeiras;
- definição dos relacionamentos e cardinalidades;
- estabelecimento de regras de integridade referencial;
- especificação de estratégias de indexação e otimização;
- preparação do modelo para implementação com JPA/Hibernate;
- definição da estratégia de evolução do esquema utilizando Flyway.

Esses elementos garantem que a estrutura de dados acompanhe as necessidades funcionais e não funcionais da aplicação.

10.3 Benefícios da Modelagem Adotada

A abordagem adotada proporciona diversas vantagens para o projeto.

Organização do domínio

A divisão das entidades por módulos facilita a compreensão do sistema e reduz o acoplamento entre diferentes áreas funcionais.

Consistência dos dados

A utilização de chaves primárias, chaves estrangeiras, restrições de unicidade e validações assegura a integridade das informações armazenadas.

Escalabilidade

O modelo foi concebido para permitir a expansão do sistema com a inclusão de novos módulos e funcionalidades sem necessidade de reestruturações significativas.

Manutenibilidade

A padronização de nomenclaturas, convenções e regras de modelagem simplifica a manutenção e reduz a probabilidade de inconsistências ao longo da evolução do projeto.

Integração tecnológica

A modelagem encontra-se totalmente alinhada às tecnologias adotadas:

- PostgreSQL 17;
- Java 21;
- Spring Boot 3;
- Spring Data JPA;
- Hibernate 6;
- Flyway.

10.4 Integração com a Arquitetura do Sistema

O modelo de dados foi concebido para integrar-se de forma transparente às demais camadas da arquitetura do GTP.

Essa integração ocorre da seguinte forma:

Camada	Integração
--------	------------

Banco de Dados	Implementação física das tabelas, índices e restrições.
----------------	---

Persistência	Entidades JPA/Hibernate e repositórios Spring Data.
--------------	---

Aplicação	Casos de uso e serviços que manipulam as entidades.
-----------	---

API REST	Exposição dos dados por meio de DTOs e endpoints.
----------	---

Frontend	Consumo das informações via API REST.
----------	---------------------------------------

Segurança	Auditoria, autenticação e autorização integradas ao modelo de dados.
-----------	--

Essa estrutura favorece a separação de responsabilidades e a evolução independente das camadas.

10.5 Governança do Modelo de Dados

A evolução do modelo deverá seguir um processo controlado, baseado nas diretrizes estabelecidas neste documento.

Toda alteração estrutural deverá:

- ser registrada em migração Flyway;
- manter compatibilidade com a versão vigente da aplicação;
- preservar a integridade dos dados;
- ser validada em ambiente de desenvolvimento e homologação antes da implantação em produção.

Esse processo garante rastreabilidade e reduz riscos associados à evolução do banco de dados.

10.6 Recomendações para Implementação

Durante a implementação do modelo recomenda-se observar as seguintes práticas:

- manter sincronização entre DER, entidades JPA e scripts Flyway;
- utilizar convenções padronizadas de nomenclatura;
- implementar auditoria em entidades críticas;
- adotar exclusão lógica (*soft delete*) para registros de negócio;
- utilizar carregamento LAZY como padrão para relacionamentos;
- criar índices apenas quando justificados por análises de desempenho;

- monitorar continuamente a evolução da base de dados.

Essas recomendações contribuem para a estabilidade e a sustentabilidade da solução.

10.7 Evolução Futura

A arquitetura do modelo foi planejada para permitir a incorporação de novas funcionalidades sem comprometer sua consistência.

Entre as possíveis evoluções destacam-se:

- georreferenciamento de territórios;
- integração com serviços de mapas;
- sincronização com aplicações móveis;
- notificações em tempo real;
- painéis analíticos avançados;
- integração com serviços externos;
- mecanismos de importação e exportação de dados;
- recursos adicionais de auditoria e conformidade.

Todas as novas entidades deverão seguir os princípios de modelagem estabelecidos neste documento.

10.8 Relação com os Demais Documentos da Arquitetura

O Documento 11 está diretamente integrado aos demais documentos do projeto.

Documento	Finalidade
Documento 03 – Regras de Negócio	Define as regras aplicáveis às entidades.
Documento 04 – Casos de Uso	Identifica as operações realizadas sobre os dados.
Documento 05 – Arquitetura do Sistema	Estabelece a organização geral da solução.
Documento 06 – Arquitetura Backend	Implementa as entidades, repositórios e serviços.
Documento 07 – Arquitetura Frontend	Consome os dados por meio da API REST.
Documento 08 – Segurança	Define auditoria, autenticação e autorização.

Documento	Finalidade
Documento 09 – Docker	Disponibiliza a infraestrutura para o PostgreSQL e Flyway.
Documento 10 – API REST	Expõe os recursos correspondentes às entidades do modelo.
Documento 12 – Banco de Dados PostgreSQL	Detalhará a implementação física do esquema de banco de dados.

Essa integração assegura a coerência entre a documentação e a implementação do sistema.

10.9 Considerações Finais

O Modelo Entidade-Relacionamento do GTP representa um dos pilares da arquitetura da plataforma, traduzindo os requisitos funcionais e as regras de negócio em uma estrutura de dados consistente e preparada para evolução.

A modelagem foi elaborada com foco em:

- clareza;
- modularidade;
- desempenho;
- segurança;
- integridade dos dados;
- facilidade de manutenção;
- escalabilidade.

A utilização de PostgreSQL, Spring Data JPA, Hibernate e Flyway, aliada às diretrizes apresentadas neste documento, oferece uma base tecnológica robusta para suportar o crescimento contínuo da aplicação.

Encerramento do Documento

Com a conclusão deste documento, o projeto GTP passa a contar com um **Modelo Entidade-Relacionamento completo**, cobrindo desde os princípios de modelagem até as estratégias de implementação, otimização e evolução do banco de dados.

Este documento servirá como referência oficial para:

- implementação das entidades Java;
- criação das tabelas PostgreSQL;

- desenvolvimento dos repositórios Spring Data JPA;
- elaboração das migrações Flyway;
- construção da API REST;
- integração com o frontend React;
- manutenção e evolução futura da plataforma.

A adoção das diretrizes aqui estabelecidas garantirá que a camada de persistência permaneça consistente, escalável e alinhada às melhores práticas de arquitetura de software durante todo o ciclo de vida do **Gestor de Territórios e Publicações (GTP)**.